

GrovLock: Design and Implementation of a Hybrid Encryption Algorithm for Secure Communications

Alegre Castilla, Sthefany Alexandra*

Espino Veas, Karla Daniela*

Panclas Aliaga, María Claudia*

Pareja Abarca, María Julia*

*Universidad Católica de Santa María

Draft / Preprint. This document is a draft under active revision. Content, figures, and conclusions may change before an eventual final version. Comments and corrections are welcome via GitHub Issues. xd

Abstract—This work presents GrovLock, a hybrid cryptographic scheme designed to protect telematic communications through the integration of key exchange mechanisms, cryptographic derivation, data diffusion, and authenticated encryption. The proposed architecture combines X25519 (ECDH) for the secure generation of shared secrets, a SHA-256 based key derivation function to obtain session keys, and AES-256-GCM to guarantee confidentiality, integrity, and authenticity. As its main contribution, GrovLock incorporates a reversible diffusion layer inspired by the mathematical principle of Grover’s diffusion operator, adapted to a classical environment. This layer transforms 128-bit blocks before encryption, increasing data dispersion and reducing identifiable patterns. The algorithm was implemented in Python and evaluated on a TCP/IP client-server architecture and on a Telegram bot. The results demonstrated secure key exchange, reliable message transmission, and correct information recovery, validating the feasibility of the proposal for secure telematic communications.

Index Terms—hybrid cryptography, ECDH, X25519, AES-256-GCM, Grover diffusion operator, authenticated encryption, key derivation function.

I. INTRODUCTION

Digital transformation has significantly increased the exchange of information through messaging applications, client-server systems, and distributed services connected through public networks, and as a consequence, protecting the confidentiality, integrity, and authenticity of transmitted data has become a fundamental requirement to guarantee the security of telematic communications. Although modern cryptographic algorithms offer high levels of protection, the emergence of new threats and the advancement of quantum computing have driven research into innovative mechanisms capable of strengthening existing security schemes.

In recent years, various studies have proposed hybrid cryptographic architectures aimed at strengthening the security of digital communications through the integration of key exchange mechanisms, cryptographic derivation functions, and authenticated encryption algorithms. According to Luc et al. [1], the incorporation of post-quantum cryptography techniques in messaging applications improves the protection of information against emerging threats derived from the ad-

vancement of quantum computing, while maintaining adequate levels of performance.

Similarly, Fauziyah, Wang, and Tabassum [2] note that multilayer protocols that combine different cryptographic mechanisms offer greater resistance against interception, manipulation, and unauthorized access attacks, since each component contributes to protecting a specific phase of the communication process. In this way, these approaches have driven the development of hybrid models capable of leveraging the advantages of asymmetric algorithms for secure key exchange and symmetric algorithms for efficient data encryption, establishing themselves as a promising alternative to guarantee the security of modern telematic communications.

Likewise, authenticated encryption through AES-GCM, or Advanced Encryption Standard, Galois/Counter Mode, continues to be one of the most widely used technologies for protecting digital communications due to its ability to simultaneously provide confidentiality, integrity, and authentication of transmitted data. According to Mandal et al. [3], AES-GCM based schemes offer an efficient combination of cryptographic security and computational performance, which is why they have been widely adopted in modern secure communication protocols, cloud services, messaging applications, and data transmission systems. Unlike conventional encryption mechanisms that only protect the content of the information, AES-GCM incorporates authentication tags that allow any unauthorized modification made during transmission to be detected, significantly reducing the risk of manipulation or message alteration attacks. On the other hand, its design based on parallelizable operations favors an efficient implementation in both software and hardware, allowing high levels of performance to be maintained even in environments with large volumes of traffic; thus, these characteristics have established AES-GCM as one of the most reliable standards for guaranteeing the security of modern telematic communications.

In parallel, the advancement of quantum computing has driven the study of algorithms capable of significantly transforming traditional information processing and security paradigms. Among these, Grover’s search algorithm stands out, originally proposed to accelerate the location of elements within unstructured databases through an iterative process of quantum amplitude amplification. According to Yoon et al. [4], the mathematical properties derived from this algorithm have sparked growing interest in the scientific community due to its potential application in new security and information process-

ing proposals; however, it is important to note that Grover's algorithm does not constitute an encryption mechanism nor was it designed to protect data, but rather to optimize search processes in quantum systems.

In this context, recent research such as that of Al-Bayaty and Perkowski [5] has analyzed the characteristics of Grover's diffusion operator, highlighting its capacity to perform reversible transformations based on reflections about the mean of a set of values. This property allows the distribution of data to be modified while preserving the information necessary to recover the original state, opening the possibility of adapting this mathematical principle to new cryptographic architectures. Consequently, the use of mechanisms inspired by Grover's diffusion operator represents an innovative alternative for increasing the transformation and dispersion of data before applying conventional encryption processes, potentially contributing to strengthening security in telematic communications.

In this context, the present work proposes GrovLock, a hybrid cryptographic scheme designed to protect telematic communications through the integration of X25519 (ECDH) for secure key exchange, a key derivation function (KDF), and AES-256-GCM for authenticated data encryption. As its main contribution, GrovLock incorporates a reversible diffusion layer inspired by Grover's diffusion operator, adapted to a classical environment and applied to 128-bit blocks before encryption. In this way, the transformation and dispersion of data is increased, strengthening the protection of information transmitted in client-server environments and messaging systems.

II. METHODOLOGY

A. Design of the GrovLock algorithm

The proposed methodology consists of the design and implementation of GrovLock, as we have named it, a hybrid cryptographic scheme oriented toward the protection of telematic communications. The architecture integrates asymmetric and symmetric cryptography mechanisms with a diffusion layer inspired by Grover's diffusion operator.

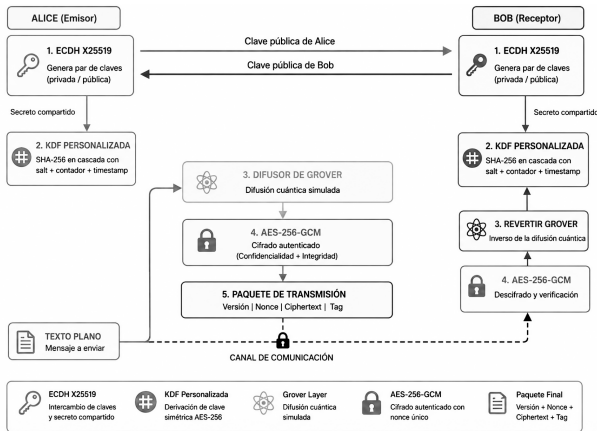


Fig. 1. Proposed architecture

The objective is to increase the transformation of data before the encryption process, adding an additional diffusion stage without affecting the recoverability of the original information. The algorithm is composed of four main stages: secure key exchange, session key derivation, data diffusion, and authenticated encryption. Fig. 1 shows the general proposed architecture.

1) *Key exchange:* The first stage of GrovLock consists of establishing a shared secret through the X25519 protocol, an implementation of the Elliptic Curve Diffie-Hellman (ECDH) mechanism based on the Curve25519 elliptic curve. According to Nguyen et al. [6], this protocol allows two entities communicating over an insecure network to independently generate the same secret key without needing to transmit it directly.

To achieve this, both the sender and the receiver generate a key pair composed of a private key and a public key; subsequently, they exchange only their public keys through the communication channel. Using their own private key and the other party's public key, each participant executes the ECDH operation to compute an identical shared secret. Due to the mathematical properties of elliptic curve cryptography, both ends obtain the same result without exposing their private keys or the derived session key.

```

37 #
38 # FASE 1: CLAVES ECDH (X25519 / Curva25519)
39 #
40
41 class ParDeClaves:
42     def __init__(self):
43         self.clave_privada = X25519PrivateKey.generate()
44         self.clave_publica = self.clave_privada.public_key()
45
46     def exportar_publica_b64(self) -> str:
47         raw = self.clave_publica.public_bytes(Encoding.Raw, PublicFormat.Raw)
48         return base64.b64encode(raw).decode()
49
50     @staticmethod
51     def importar_publica(b64: str) -> X25519PublicKey:
52         raw = base64.b64decode(b64)
53         return X25519PublicKey.from_public_bytes(raw)

```

Fig. 2. Code for X25519 keys

The exchange() operation allows both participants to obtain the same shared secret without transmitting it directly over the network.

```

secreto_ecdh = self._par.clave_privada.exchange(pub_remota)

```

Fig. 3. Exchange operation

The use of X25519 provides significant advantages in terms of security and performance, as it offers resistance against known attacks on key exchange, reduces computational cost compared to other traditional asymmetric schemes, and is widely used in modern secure communication protocols. In GrovLock, the shared secret generated through X25519 forms the basis for the derivation of the session key subsequently used by the diffusion layer and the AES-256-GCM encryption process.

2) *Key derivation:* Once the shared secret is established through X25519, it is not used directly as the encryption key. Instead, it is processed through a key derivation function (KDF) based on SHA-256 with the purpose of generating a

cryptographically robust session key suitable for the subsequent stages of the algorithm. KDF functions transform initial cryptographic material into high-entropy pseudorandom keys, reducing possible biases present in the input and guaranteeing properties appropriate for use in encryption and authentication mechanisms.

According to Chuah, Harun, and Hamid [7], KDFs constitute a fundamental component of modern security protocols, as they allow one or more cryptographic keys to be generated from shared input material of arbitrary length. Likewise, Bhati et al. [8] highlight that HKDF is currently one of the most widely used derivation mechanisms in key exchange protocols and secure messaging systems due to its extract-then-expand approach, which produces uniformly distributed keys resistant to cryptographic attacks.

In GrovLock, the KDF generates a 256-bit session key, subsequently used both to parameterize the Grover-inspired diffusion layer and for the authenticated encryption process via AES-256-GCM.

```

56 # -----
57 # FASE 2: KDF PERSONALIZADA
58 # -----
59
60 def kdf_propio(secreto: bytes, contador: int) -> bytes:
61     """
62     SHA256( secreto || SALT || SHA256( contador || timestamp ) )
63     Produce 32 bytes + clave AES-256.
64     """
65     ts_bytes = struct.pack(">Q", int(time.time()))
66     cnt_bytes = struct.pack(">Q", contador)
67     inner = hashlib.sha256(cnt_bytes + ts_bytes).digest()
68     return hashlib.sha256(secreto + SALT_PERSONALIZADO + inner).digest()
69
70
71 def generar_nonce(contador: int) -> bytes:
72     """SHA256( microsegundos || contador || random )[:12]"""
73     ts = struct.pack(">Q", int(time.time() * 1_000_000))
74     cnt = struct.pack(">Q", contador)
75     rnd = os.urandom(4)
76     return hashlib.sha256(ts + cnt + rnd).digest()[:12]

```

Fig. 4. Code for the KDF

3) *Diffusion layer*: The main contribution of GrovLock is the incorporation of a reversible diffusion layer inspired by Grover's diffusion operator; it is important to note that the proposal does not implement Grover's quantum search algorithm nor does it require quantum computing infrastructure. Instead, it adopts only the mathematical principle of reflection about the mean and adapts it to a classical data processing environment with the objective of increasing the dispersion of information before the encryption process.

Diffusion is applied over 128-bit (16-byte) blocks, and for each block, the mean of the values that compose it is first computed, and each byte is subsequently transformed through a reflection operation about that mean. The transformation used is defined as:

$$D(v[i]) = (2\mu - v[i]) \bmod 256 \quad (1)$$

where $v[i]$ represents the original value of byte i , μ corresponds to the mean of all bytes in the block, and $D(v[i])$ represents the value obtained after applying diffusion; in this way, this operation generates a redistribution of the values

within the block, increasing data dispersion and reducing the presence of identifiable patterns before the encryption stage.

According to Al-Bayaty and Perkowski [5], the mathematical principles associated with Grover's diffusion operator can be adapted to perform reversible transformations on data sets; based on this concept, GrovLock implements a classical variant of this transformation as an additional mixing mechanism prior to authenticated encryption. Likewise, the number of diffusion rounds is dynamically derived from the session key generated through X25519 and the KDF function. This introduces variability between different communication sessions, such that similar messages may undergo different transformation processes; finally, because the operation is reversible, the receiver can recover the exact original data before decryption, preserving the integrity of the transmitted information.

In essence, rather than executing the full quantum algorithm, the reflection-about-the-mean operator was mathematically adapted to operate on blocks of bytes.

```

39 def _difusor_grover_bloque(bloque: List[int], rondas: int) -> List[int]:
40     """
41     Aplica el difusor de Grover a un bloque de bytes.
42
43     Para garantizar invertibilidad exacta sin pérdidas de redondeo,
44     calculamos la suma (en vez del promedio) y operamos con enteros:
45
46     suma = Σ v[i]
47     w[i] = (2·suma//N - v[i]) mod 256
48
49     Esto es equivalente a D(v)[i] = (2μ - v[i]) mod 256 pero con
50     aritmética entera exacta cuando N divide a 2·suma.
51     """
52     v = list(bloque)
53     n = len(v)
54
55     for _ in range(rondas):
56         S = sum(v) # Suma total (entero exacto)
57         # 2·μ·N = 2·S/N·N = 2S, pero necesitamos 2·μ = 2S/N
58         # Usamos: w[i] = (2*S - N*v[i]) // N si N | (2S - N*v[i])
59         # Más simple: trabajamos con escala N para mantener enteros
60         doble_media_n = 2 * S # = 2·μ·N (sin dividir)
61         v = [(doble_media_n - n * vi) % (256 * n) // n for vi in v]
62         # Simplificado: (2S/N - vi) mod 256 con aritmética entera
63         v = [(2 * S // n - vi) % 256 for vi in v]
64
65     return v

```

Fig. 5. Code for the block diffuser

4) *Authenticated encryption*: Once the diffusion layer has been applied, the data is encrypted using AES-256-GCM, using the session key derived during key exchange. This algorithm was selected because it simultaneously provides confidentiality, integrity, and authenticity of the transmitted information. In addition, it incorporates verification mechanisms that allow any unauthorized modification of the data during transmission to be detected. In this way, GrovLock guarantees that only users who share the session key can access the original content of the messages.

According to Mandal et al. [3], AEAD schemes, or Authenticated Encryption with Associated Data, based on AES-GCM continue to be one of the most robust and widely adopted alternatives for protecting digital communications due to their balance between cryptographic security, computational efficiency, and the ability to detect unauthorized modifications. For this reason, GrovLock uses AES-256-GCM as the final protection mechanism, complementing the prior diffusion of

```

83 def cifrar(clave: bytes, mensaje: str, contador: int) -> str:
84
85     # Fase 3: capa Grover
86     datos_planos = mensaje.encode("utf-8")
87     datos_grover = aplicar_grover(datos_planos, clave)
88
89     # Fase 4: AES-256-GCM
90     nonce = generar_nonce(contador)
91     aesgcm = AESGCM(clave)
92     cifrado = aesgcm.encrypt(nonce, datos_grover, associated_data=None)
93
94     # Empaquetar: versión(1) + nonce(12) + ciphertext+tag
95     paquete = VERSION_PROTOCOLO + nonce + cifrado
96     return base64.b64encode(paquete).decode()

```

Fig. 6. Encryption function

```

110 def descifrar(clave: bytes, b64_texto: str) -> str:
111
112     paquete = base64.b64decode(b64_texto)
113
114     version = paquete[1]
115     nonce = paquete[1:13]
116     cifrado = paquete[13:]
117
118     # Fase 4 inversa: AES-GCM
119     aesgcm = AESGCM(clave)
120     datos_grover = aesgcm.decrypt(nonce, cifrado, associated_data=None)
121
122     # Fase 3 inversa: revertir Grover
123     datos_planos = revertir_grover(datos_grover, clave)
124     return datos_planos.decode("utf-8")

```

Fig. 7. Decryption function

the data with an internationally recognized authenticated encryption scheme.

B. Experimental implementation

GrovLock was implemented using the Python programming language due to its wide availability of cryptographic and network communication libraries. In order to validate the algorithm's functioning in real communication scenarios, tests were carried out in different environments.

C. Telematic application, or Telegram bot

The bot implements an additional end-to-end encryption layer on top of Telegram. Telegram already encrypts its own communications with the MTProto protocol, but with our system we add a second layer: the messages that travel through Telegram's servers are Base64 ciphertext, completely unreadable to Telegram or any intermediary.

1) System components:

- **bot.py**: the bot server running on the developer's laptop. Registers handlers for each command and manages message delivery between users using the Telegram API.
- **session_manager.py**: manages the state of all active sessions in memory. Maintains key pairs, pending connection requests, derived session keys, and message counters.
- **hybrid_cipher_v2.py + grover_layer.py**: the cryptographic core, invoked by the manager on every encryption or decryption operation.

2) *Complete protocol flow on Telegram*: The following is the step-by-step flow of an encrypted session between two Telegram users:

- 1) Both users open the bot and run `/miid` to obtain their numeric Telegram User ID.

TABLE I
BOT COMMANDS

Command	Description
<code>/start</code>	Welcome message and bot instructions
<code>/miid</code>	Shows your Telegram User ID (number required to connect)
<code>/nuevasesion</code>	Generates an X25519 key pair and starts the handshake process
<code>/conectar</code>	<code>/conectar ;id_i</code> — Requests an encrypted connection with another user
<code>/aceptar</code>	<code>/aceptar ;id_i</code> — Accepts the request and completes the ECDH handshake
<code>/enviar</code>	<code>/enviar ;message_i</code> — Encrypts with Grover+AES and delivers to the recipient
<code>/descifrar</code>	<code>/descifrar ;base64_i</code> — Manually decrypts a received text
<code>/estado</code>	Shows session status: partner, derived key, messages sent

- 2) Both run `/nuevasesion`. The bot internally creates a `SesionCifradoV2` object for each one and generates an X25519 key pair. The private key never leaves the server.
- 3) User A runs `/conectar ;idB`. The bot registers the request in the `solicitudes_pendientes` dictionary and sends an automatic notification to user B.
- 4) User B runs `/aceptar ;idA`. The bot calls `SessionManager.completar_handshake()`, which executes the ECDH exchange: each session uses the other's public key to compute the shared secret, then `kdf_propio()` derives the symmetric key.
- 5) Both users receive a confirmation message that includes the first bytes of the derived key (identical on both sides, visually verifiable).
- 6) From this point on, any text typed by user A (via `/enviar` or directly) passes through the `cifrar()` pipeline before being transmitted.
- 7) The bot delivers the message to user B and simultaneously calls `descifrar()` with B's session to show them the original text along with the Base64 that traveled through Telegram.

D. Encrypted client/server chat over WiFi

1) *Network architecture*: The WiFi client/server system demonstrates the protocol over real TCP/IP without depending on any third-party service. Messages travel over the local network as ciphertext, and any network analysis tool such as Wireshark would only see the unreadable Base64.

- **servidor.py**: runs on the laptop. Listens on all network interfaces (0.0.0.0) on port 8888. On startup it automatically detects the local WiFi IP and displays it. Accepts multiple clients with one thread per connection.
- **cliente.py**: runs on any other device on the network. Receives the server's IP as a command-line argument. Establishes a TCP connection and automatically runs the ECDH handshake.

2) *JSON message protocol*: The two processes communicate using newline-delimited JSON. Each message has a type field indicating its purpose:


```
{ "tipo": "handshake_pub",
  "pub_key": "BASE64_32_bytes..." }
{ "tipo": "handshake_ok",
  "msg": "Sesion cifrada establecida" }
{ "tipo": "mensaje",
  "cifrado": "BASE64_cifrado..." }
{ "tipo": "fin" }
```

3) *Handshake flow over a real network*: Establishing a secure session in GrovLock is done through a key exchange process based on X25519 over a TCP connection, and this procedure allows both parties to generate a shared secret for deriving the keys used during encrypted communication.

- 1) The client opens a TCP connection to the server (socket.connect)
- 2) The client sends its X25519 public key in JSON format
- 3) The server responds with its own X25519 public key
- 4) Both call `completar_handshake()` with the received key, using the same `session_id=1337`
- 5) The server sends `handshake_ok` as confirmation that the session is ready
- 6) From this point, each message passes through `cifrar()` before being sent and `descifrar()` upon receipt
- 7) An independent thread (`recibir_loop`) listens for incoming messages without blocking user input.

E. Evaluation

The first environment consisted of a client-server architecture based on TCP sockets, where the client establishes a connection with the server, executes the key exchange via X25519, derives the corresponding session key, applies the GrovLock diffusion layer to the data, and finally encrypts the information using AES-256-GCM before transmission. Once the message is received, the server verifies the authenticity of the data, executes the decryption process, and applies the inverse transformation of the diffusion layer to recover the original message.

As a second experimental environment, a Telegram bot was developed that fully integrates the GrovLock scheme into the platform's communication flow. The system allows two users to establish an encrypted session through a key exchange process based on X25519, generating a shared session key from which the cryptographic parameters used by the algorithm are derived. Once the session establishment process is complete, messages are subjected to the Grover-inspired diffusion layer and subsequently encrypted via AES-256-GCM before being sent through Telegram's infrastructure. As a result, the platform only transports Base64-encoded ciphertext strings, while the original content remains protected by the proposed cryptographic scheme. At the receiving end, the message is authenticated, decrypted, and automatically reconstructed using the same shared session key.

The implementation also incorporates functionality for dynamic session generation, connection establishment between users, secure key exchange, and automatic message encryption and decryption, allowing GrovLock's behavior to be evaluated in a widely used instant messaging environment.

III. RESULTS

With the objective of validating the functioning of the proposed algorithm in a real communication environment, a telematic application based on the Telegram platform was developed; Telegram was chosen because it provides a widely used messaging infrastructure, allowing the behavior of the cryptographic scheme to be evaluated under real usage conditions and over public Internet networks.

It is important to note that Telegram already implements its own data protection mechanisms through the MTProto protocol; however, in this work an additional cryptographic layer was added that operates independently of the security provided by the platform. In this way, messages are encrypted by the HybridECDH-Grover-AES algorithm before being sent to Telegram, achieving an application-level end-to-end encryption scheme.

As a result, Telegram's servers only transport Base64-encoded strings containing previously encrypted information, which means that the original content of the message can only be recovered by users who have correctly completed the key exchange process and share the same derived session key.

A. Main system components

The developed architecture is composed of three main modules.

1) *Communication module (bot.py)*: This component constitutes the point of interaction between users and the system. It is responsible for registering available commands, receiving requests, coordinating message exchange, and communicating with the official Telegram API. It also acts as an intermediary between users and the cryptographic module.

```
474 def main():
475     print("\n" * 60)
476     print(" HybridECDH-Grover-AES Bot - Iniciando...")
477     print("\n" * 60)
478
479     app = Application.builder().token(TOKEN).build()
480
481     # Registrar handlers
482     app.add_handler(CommandHandler("start", cmd_start))
483     app.add_handler(CommandHandler("mid", cmd_mid))
484     app.add_handler(CommandHandler("nuevasesion", cmd_nueva_sesion))
485     app.add_handler(CommandHandler("conectar", cmd_conectar))
486     app.add_handler(CommandHandler("aceptar", cmd_aceptar))
487     app.add_handler(CommandHandler("enviar", cmd_enviar))
488     app.add_handler(CommandHandler("descifrar", cmd_descifrar))
489     app.add_handler(CommandHandler("estado", cmd_estado))
490     app.add_handler(CommandHandler("ayuda", cmd_ayuda))
491     app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, mensaje_libre))
492
493     print(" Bot corriendo. Presiona Ctrl+C para detener.")
494     print("\n" * 60)
495     app.run_polling(allowed_updates=Update.ALL_TYPES)
496
497
498 if __name__ == "__main__":
499     main()
```

Fig. 8. Code for the communication module

2) *Session management module (session_manager.py)*: This module manages all information related to active sessions, and among its responsibilities are session generation, temporary storage of connection requests, association between connected users, and management of the derived keys obtained during the ECDH process; additionally, it maintains the message counters necessary to guarantee the uniqueness of the nonces used by AES-GCM.

```

16 class EstadoSesion:
17     """Estado de la sesión entre dos usuarios."""
18     ESPERANDO_HANDSHAKE = "esperando_handshake"
19     ACTIVA = "activa"
20
21     def __init__(self, owner_id: int):
22         self.owner_id = owner_id
23         self.sesion = SesiónCifradoV2(str(owner_id))
24         self.estado = self.ESPERANDO_HANDSHAKE
25         self.partner_id: Optional[int] = None
26
27     @property
28     def lista(self) -> bool:
29         return self.estado == self.ACTIVA

```

Fig. 9. Key code for the session management module

3) *Cryptographic core (hybrid_cipher_v2.py and grover_layer.py)*: This component implements the entirety of the proposed algorithm, including X25519 key pair generation, SHA-256 based key derivation, the Grover-inspired diffusion layer, and authenticated encryption via AES-256-GCM; also, every time a user sends or receives a message, this module automatically executes the cryptographic operations necessary to guarantee the confidentiality and integrity of the information.

```

1  ** **
2
3  CAPA DE GROVER - Difusor cuántico simulado
4
5  Operador de difusión:  $D(v)[i] = (2\mu - v[i]) \bmod 256$ 
6  donde  $\mu = \text{mean}(v)$  - basado en Grover (1996)
7

```

Fig. 10. Functionality of the Grover layer

```

1  ** **
2
3  HybridECDH-Grover-AES - Versión 2.0
4
5  Fase 1 - ECDH X25519 : intercambio de claves
6  Fase 2 - KDF personalizada : derivación SHA-256 cascada
7  Fase 3 - Difusor de Grover : difusión cuántica simulada
8  Fase 4 - AES-256-GCM : cifrado autenticado
9
10 ** **

```

Fig. 11. Phases of HybridECDH-Grover-AES

B. Complete protocol flow on Telegram

The secure communication process developed can be divided into several consecutive stages.

1) *User identification*: Initially, both participants obtain their unique Telegram identifier through the `/miid` command. This identifier subsequently allows cryptographic sessions to be associated with specific users within the system.

2) *Key generation*: Each user then runs the `/nuevasesion` command. At this stage the system internally creates a `SesiónCifradoV2` type object, which automatically generates an X25519 cryptographic key pair composed of a private key and a public key. The private key remains stored exclusively within the corresponding session and is never transmitted or shown to the user.

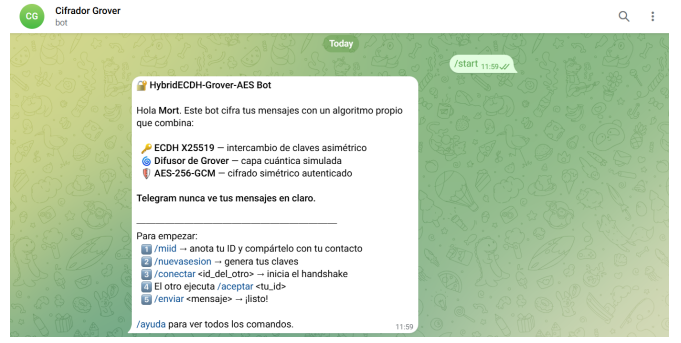


Fig. 12. Start of conversation on Telegram

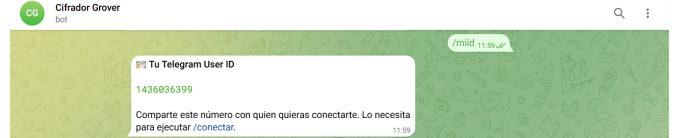


Fig. 13. Generation of a User ID on Telegram

3) *Connection request*: Once the keys have been generated, the user who wishes to initiate communication runs the command: `/conectar <destination_id>`; in this way, the system registers the request and automatically notifies the destination user that there is a pending encrypted connection request.



Fig. 14. Message with public key on Telegram

4) *ECDH exchange and key derivation*: When the second user accepts the request via: `/aceptar <origin_id>`, the system automatically executes the ECDH protocol; during this stage both sessions exchange only their public keys and locally compute the same shared secret using their respective private keys. This secret is subsequently processed by the SHA-256 based key derivation function to produce a 256-bit symmetric key; this key will then be used throughout the session to encrypt and decrypt messages.

5) *Transmission of encrypted messages*: Once key exchange is complete, users can begin exchanging messages. Each message sent goes through the following stages:

6) *Message exchange*: Multiple test messages were subsequently sent in both directions; it was observed that Telegram only transported Base64-encoded strings generated by the encryption process, while the original content was correctly recovered only by the authorized recipient. It should be noted that during testing no information loss or inconsistencies were detected in the encryption and decryption process.

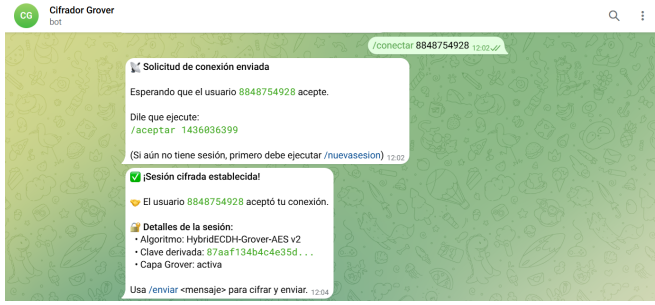


Fig. 15. Connection with the User ID

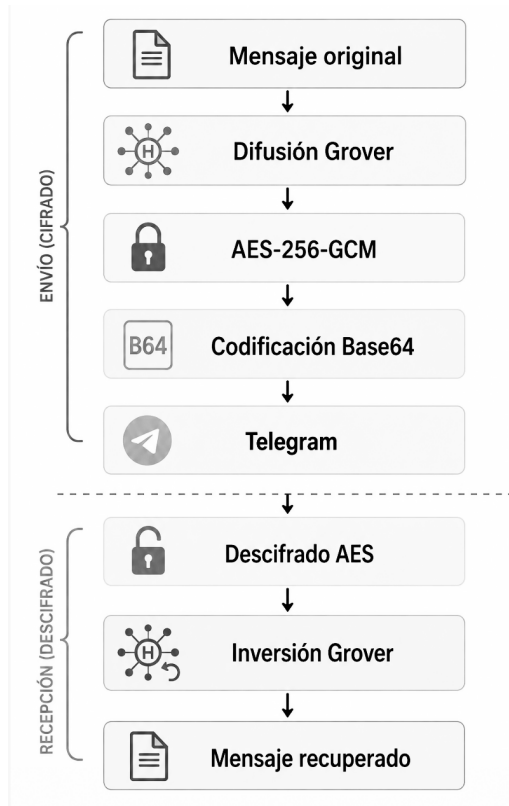


Fig. 16. Flow of the stages

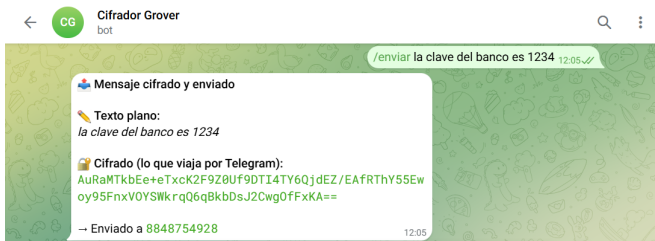


Fig. 17. Encrypted message sent



Fig. 18. Message decrypted from another mobile device

IV. DISCUSSION

The results obtained demonstrate that GrovLock can be successfully implemented in real telematic communication environments through the integration of X25519 key exchange, SHA-256 based key derivation, a Grover-inspired diffusion layer, and AES-256-GCM authenticated encryption; particularly in the messaging application developed on Telegram, the algorithm allowed secure sessions to be established, keys to be exchanged transparently, and original messages to be correctly recovered after the decryption process.

The incorporation of a diffusion layer prior to encryption constitutes the main contribution of the proposal. Unlike conventional hybrid schemes, which generally combine only key exchange and authenticated encryption mechanisms, GrovLock introduces a reversible transformation inspired by Grover's diffusion operator, and this stage increases data dispersion prior to the application of AES-256-GCM, reducing the presence of recognizable patterns in plaintext and adding an additional transformation layer. Although this operation

does not replace the cryptographic guarantees provided by AES, it can contribute to increasing the structural complexity of the processed data.

The results also show that the use of X25519 allowed shared secrets to be generated efficiently and securely without transmitting sensitive information over the network. This behavior is consistent with what was reported by Nguyen et al. [6], who highlight the efficiency and security of Curve25519 in modern key exchange protocols. Likewise, the use of AES-256-GCM allowed the confidentiality, integrity, and authenticity of transmitted messages to be simultaneously guaranteed, in agreement with the findings reported by Mandal et al. [3] regarding the robustness of AEAD schemes in secure communication applications.

For its part, the implementation on Telegram allowed the practical viability of the algorithm to be verified on widely used messaging platforms; thus, transmitted messages were observed only as Base64-encoded ciphertext strings, while the original content remained accessible only to users who shared the same session key. This behavior demonstrates that GrovLock can function as a complementary cryptographic layer independent of the security mechanisms already provided by the platform.

As future work, it is proposed to carry out comparative performance evaluations against traditional hybrid schemes, incorporate statistical analysis metrics on the transformed data, and develop deeper cryptographic studies to determine the real impact of the Grover-inspired diffusion layer on system security. In addition, it would be interesting to explore adapting the proposal to large-scale communication environments, IoT applications, and distributed architectures where information protection is a critical requirement.

V. CONCLUSIONS

This work made it possible to design and implement GrovLock, a hybrid cryptographic scheme oriented toward the protection of telematic communications; the integration of X25519 for key exchange, a SHA-256 based derivation function, a Grover-inspired diffusion layer, and AES-256-GCM allowed secure sessions to be established and effectively protected the information exchanged between users. In this way, the incorporation of a reversible diffusion layer inspired by Grover's operator constitutes the main contribution of the proposal. This transformation increases data dispersion prior to the encryption process, adding an additional mixing stage without affecting the ability to recover the original message or compatibility with widely used cryptographic algorithms.

The implementation of the algorithm in a Telegram bot allowed its applicability to be validated in a real instant messaging environment. The results showed that messages sent through the platform were transported only as ciphertext, while the original content remained accessible exclusively to authorized users who shared the same session key. For this reason, the use of AES-256-GCM as an authenticated encryption mechanism allowed confidentiality, authentication,

and integrity of transmitted data to be simultaneously provided. In this way, the combination of this algorithm with the proposed diffusion stage and secure key exchange produced a multilayer architecture capable of strengthening the protection of telematic communications against various threats.

In summary, the development of GrovLock made it possible to demonstrate the feasibility of integrating different cryptographic mechanisms into a hybrid architecture oriented toward the protection of telematic communications; the combination of secure key exchange, cryptographic derivation, data diffusion, and authenticated encryption enabled the construction of a functional system capable of guaranteeing the confidentiality, integrity, and authenticity of transmitted information. Likewise, implementation and validation in real environments demonstrated the potential of this type of proposal to strengthen the security of digital communications and to serve as a basis for future research in the field of applied cryptography.

REFERENCES

- [1] Nhu-Quynh Luc, Tat-Thang Nguyen, and Thanh Toan Dao, "Developing Secure Messaging Software Using Post-Quantum Cryptography," *Engineering, Technology & Applied Science Research*, vol. 13, no. 6, pp. 12440–12445, 2023, doi: 10.48084/etasr.6549.
- [2] Fauziyah, Z. Wang, and M. Tabassum, "A Holistic Secure Communication Mechanism Using a Multilayered Cryptographic Protocol to Enhanced Security," *Computers, Materials & Continua*, vol. 78, no. 3, 2024, doi: 10.32604/cmc.2024.046797.
- [3] Surajit Mandal, Ravi Anand, Mostafizar Rahman, Santanu Sarkar, and Takanori Isobe, "Implementing Grover's on AES-Based AEAD Schemes," *Sci. Rep.*, vol. 14, 2024, doi: 10.1038/s41598-024-69188-8.
- [4] Chun Seok Yoon, Chang Ho Hong, Min Sung Kang, Ji-Woong Choi, and Hyung Jin Yang, "Quantum Asymmetric Key Crypto Scheme Using Grover Iteration," *Sci. Rep.*, vol. 13, 2023, doi: 10.1038/s41598-023-30860-0.
- [5] A. Al-Bayaty and M. Perkowski, "A Concept of Controlling Grover Diffusion Operator: A New Approach to Solve Arbitrary Boolean-Based Problems," *Sci. Rep.*, vol. 14, 2024, doi: 10.1038/s41598-024-74587-y.
- [6] H. Nguyen, T. Hoang, and L. Tran, "Efficient Hardware Implementation of Elliptic-Curve Diffie-Hellman Ephemeral on Curve25519," *Electronics (Basel)*, vol. 12, no. 21, p. 4480, 2023, doi: 10.3390/electronics12214480.
- [7] C. W. Chuah, N. Z. Harun, and I. R. A. Hamid, "Key Derivation Function: Key-Hash Based Computational Extractor and Stream Based Pseudorandom Expander," *PeerJ Comput. Sci.*, vol. 10, p. e2249, 2024, doi: 10.7717/peerj-cs.2249.
- [8] A. S. Bhati, A. Dufka, E. Andreeva, A. Roy, and B. Preneel, "Skye: An Expanding PRF based Fast KDF and its Applications," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (AsiaCCS '24)*, 2024, doi: 10.1145/3634737.3637673.